

Student Registration System

A Priority-Based Course Enrollment Management System Implementation Report and Technical Documentation

z0hra321

January 12, 2026

Abstract

This document presents a comprehensive technical report on the Student Registration System, a production-ready C++17 application designed for educational institutions. The system implements an intelligent priority-based waitlist mechanism using a min-heap data structure to manage course enrollments efficiently. This report covers the system architecture, algorithm design, implementation details, time complexity analysis, testing methodology, and usage documentation. The system supports dual interfaces (CLI and GUI) and demonstrates efficient algorithms with logarithmic time complexity for waitlist operations, making it suitable for large-scale deployment in academic environments.

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Problem Statement	4
1.3	Objectives	4
1.4	Scope	4
2	System Architecture	5
2.1	High-Level Design	5
2.2	Component Description	5
2.2.1	WaitlistPQ	5
2.2.2	RegistrationSystem	5
2.2.3	Presentation Layer	5
2.3	Data Structures	6
3	Algorithm Design	6
3.1	Priority Queue Implementation	6
3.2	Heap Properties	6
3.3	Priority Comparison	6
3.4	Enrollment Algorithm	7
3.5	Auto-Enrollment Algorithm	7
3.6	Heap Operations	7

3.6.1	Heapify Up	7
3.6.2	Heapify Down	8
4	Time Complexity Analysis	8
4.1	Waitlist Priority Queue Operations	8
4.2	Registration System Operations	8
4.3	Space Complexity	9
5	Implementation Details	9
5.1	Code Structure	9
5.2	Key Implementation Features	10
5.2.1	Move Semantics	10
5.2.2	Input Validation	10
5.2.3	Error Handling	11
6	Build System	11
6.1	CMake Configuration	11
6.2	Build Commands	12
7	Testing and Validation	12
7.1	Unit Testing	12
7.2	Test Results	12
8	Usage Examples	13
8.1	CLI Interface	13
8.2	GUI Interface	13
8.2.1	Interface Overview	13
8.2.2	Students Management Tab	13
8.2.3	Courses Management Tab	14
8.2.4	Registration Tab	14
8.2.5	My Courses Tab	15
8.2.6	Waitlist Management Tab	15
8.2.7	GUI Features and Benefits	16
8.2.8	User Experience	17
9	Performance Analysis	17
9.1	Benchmark Results	17
9.2	Scalability	18
10	Conclusion	18
10.1	Future Enhancements	19
11	Team Contributions	19
11.1	Project Leadership	19
11.2	Development Team	19
11.3	Collaborative Development	20
12	Acknowledgments	20
A	Source Code Repository	21

B Additional Resources**21**

1 Introduction

1.1 Project Overview

The Student Registration System is a high-performance course enrollment management solution designed to automate and streamline the registration process for educational institutions. The system addresses the critical need for fair, efficient, and automated course allocation when demand exceeds available capacity.

1.2 Problem Statement

Traditional course registration systems face several challenges:

- Manual waitlist management is time-consuming and error-prone
- Fair prioritization among students requires consistent rule application
- Real-time enrollment updates are necessary when students drop courses
- Support for both administrative (CLI) and end-user (GUI) interfaces

This system solves these challenges through an automated, priority-based enrollment mechanism.

1.3 Objectives

The primary objectives of this project are:

1. Implement an efficient priority queue for waitlist management
2. Ensure fair course allocation based on academic standing
3. Provide real-time automatic enrollment when seats become available
4. Deliver both command-line and graphical user interfaces
5. Achieve optimal time complexity for all operations

1.4 Scope

This report covers:

- System architecture and design decisions
- Data structure selection and implementation
- Algorithm design and complexity analysis
- Implementation details and code structure
- Testing methodology and validation
- Build and deployment instructions

2 System Architecture

2.1 High-Level Design

The Student Registration System follows a modular architecture with clear separation of concerns:

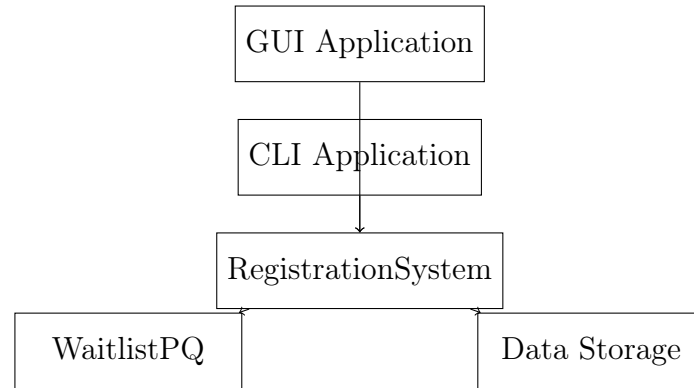


Figure 1: System Architecture Overview

2.2 Component Description

2.2.1 WaitlistPQ

The core data structure implementing a min-heap priority queue. This component manages:

- Student entries with academic level priority
- Timestamp-based tie-breaking within the same level
- Efficient insertion and extraction operations

2.2.2 RegistrationSystem

The main controller class that orchestrates:

- Student and course management
- Enrollment and drop operations
- Waitlist integration and auto-enrollment
- Statistics and query operations

2.2.3 Presentation Layer

Two interface implementations:

- **CLI**: Command-line interface for automation and testing
- **GUI**: Dear ImGui-based graphical interface for interactive use

2.3 Data Structures

The system employs the following data structures:

Structure	Type	Purpose
WaitlistPQ	Min-Heap	Priority queue for waitlist management
Student Storage	<code>std::unordered_map</code>	$O(1)$ student lookup by ID
Course Storage	<code>std::unordered_map</code>	$O(1)$ course lookup by code
Enrollment Lists	<code>std::vector</code>	Sequential storage of enrolled student IDs

Table 1: Data Structure Overview

3 Algorithm Design

3.1 Priority Queue Implementation

The waitlist system uses a binary min-heap where each node represents a student entry with:

- **Priority:** Academic level (Senior=1, Junior=2, Sophomore=3, Freshman=4)
- **Timestamp:** Order of registration for tie-breaking
- **Student ID:** Unique identifier

3.2 Heap Properties

A min-heap maintains the following invariant:

$$\text{heap}[i] \leq \text{heap}[\text{parent}(i)] \quad (1)$$

Where $\text{parent}(i) = \lfloor (i - 1)/2 \rfloor$

3.3 Priority Comparison

The comparison function determines entry ordering:

Algorithm 1 Priority Comparison Function

```

1: function HIGHERPRIORITY(entry1, entry2)
2:   if entry1.priority ≠ entry2.priority then
3:     return entry1.priority < entry2.priority
4:   else
5:     return entry1.timestamp < entry2.timestamp
6:   end if
7: end function

```

This ensures:

1. Higher academic level (lower priority number) takes precedence
2. Earlier timestamp breaks ties within the same level (FIFO)

3.4 Enrollment Algorithm

Algorithm 2 Student Enrollment Process

```

1: function ENROLL(studentId, courseCode)
2:   Validate student and course existence
3:   if student already enrolled then
4:     return ALREADYENROLLED
5:   end if
6:   if enrolledCount(course) < capacity(course) then
7:     Add student to enrolled list
8:     return ENROLLED
9:   else
10:    ADDTOWAITLIST(studentId, level(student))
11:    return WAITLISTED
12:   end if
13: end function

```

3.5 Auto-Enrollment Algorithm

Algorithm 3 Course Drop with Auto-Enrollment

```

1: function DROP(studentId, courseCode)
2:   Remove student from enrolled list
3:   if waitlist is not empty then
4:     nextStudent ← REMOVEFROMWAITLIST
5:     Add nextStudent to enrolled list
6:     return DROPPEDANDENROLLED
7:   else
8:     return DROPPED
9:   end if
10: end function

```

3.6 Heap Operations

3.6.1 Heapify Up

After inserting an element at the end, restore heap property by bubbling up:

Algorithm 4 Heapify Up

```

1: function HEAPIFYUP(index)
2:   while index > 0 and HIGHERPRIORITY(heap[index], heap[parent(index))] do
3:     SWAP(heap[index], heap[parent(index)])
4:     index ← parent(index)
5:   end while
6: end function

```

3.6.2 Heapify Down

After removing the root, restore heap property by percolating down:

Algorithm 5 Heapify Down

```

1: function HEAPIFYDOWN(index)
2:   smallest  $\leftarrow$  index
3:   left  $\leftarrow 2 \cdot \textit{index} + 1$ 
4:   right  $\leftarrow 2 \cdot \textit{index} + 2$ 
5:   if left < size and HIGHERPRIORITY(heap[left], heap[smallest]) then
6:     smallest  $\leftarrow$  left
7:   end if
8:   if right < size and HIGHERPRIORITY(heap[right], heap[smallest]) then
9:     smallest  $\leftarrow$  right
10:  end if
11:  if smallest  $\neq$  index then
12:    SWAP(heap[index], heap[smallest])
13:    HEAPIFYDOWN(smallest)
14:  end if
15: end function

```

4 Time Complexity Analysis

4.1 Waitlist Priority Queue Operations

Let n denote the number of students in the waitlist.

Operation	Time	Space	Description
addToWaitlist()	$O(\log n)$	$O(1)$	Insert entry and maintain heap
removeFromWaitlist()	$O(\log n)$	$O(1)$	Extract minimum and restore heap
autoEnroll()	$O(\log n)$	$O(1)$	Wrapper for removal
isEmpty()	$O(1)$	$O(1)$	Check emptiness
size()	$O(1)$	$O(1)$	Return size

Table 2: WaitlistPQ Complexity

Justification: Heap insertion and extraction require traversing the tree height, which is $\lfloor \log_2 n \rfloor + 1$, resulting in $O(\log n)$ complexity.

4.2 Registration System Operations

Let n = number of students, m = number of courses, and k = enrolled students in a course.

Detailed Breakdown:

- **Hash Map Operations:** Average $O(1)$ due to hash-based indexing. Worst-case $O(n)$ or $O(m)$ occurs only with hash collisions.
- **Enrollment Check:** Linear search through enrolled list: $O(k)$

Operation	Time	Space	Description
<code>addStudent()</code>	$O(1)$ avg, $O(n)$ worst	$O(1)$	Hash map insertion
<code>getStudent()</code>	$O(1)$ avg, $O(n)$ worst	$O(1)$	Hash map lookup
<code>listStudents()</code>	$O(n)$	$O(n)$	Enumerate all students
<code>addCourse()</code>	$O(1)$ avg, $O(m)$ worst	$O(1)$	Hash map insertion
<code>getCourse()</code>	$O(1)$ avg, $O(m)$ worst	$O(1)$	Hash map lookup
<code>listCourses()</code>	$O(m)$	$O(m)$	Enumerate all courses
<code>enroll()</code>	$O(k + \log n)$	$O(1)$	Enroll or waitlist
<code>drop()</code>	$O(k + \log n)$	$O(1)$	Drop and auto-enroll
<code>coursesOfStudent()</code>	$O(m \cdot k)$	$O(m)$	Find student's courses

Table 3: RegistrationSystem Complexity

- **Enrollment Removal:** Find and erase operation: $O(k)$
- **Waitlist Operations:** Priority queue operations: $O(\log n)$
- **Student Course Query:** Iterate through all courses: $O(m)$, check each enrollment list: $O(k)$ per course

4.3 Space Complexity

Total space complexity: $O(n + m \cdot k)$ where:

- n : Total number of students
- m : Total number of courses
- k : Average enrolled students per course

Breakdown:

- Student storage: $O(n)$
- Course storage: $O(m)$
- Enrollment lists: $O(m \cdot k)$
- Waitlists: $O(n)$ worst-case (all students waitlisted)

5 Implementation Details

5.1 Code Structure

The implementation follows C++17 standards with modern practices:

Listing 1: WaitlistPQ Header Interface

```

1 #pragma once
2
3 #include <string>
4 #include <vector>

```

```

5
6 namespace waitlist {
7     enum class Level {
8         SENIOR = 1,
9         JUNIOR = 2,
10        SOPHOMORE = 3,
11        FRESHMAN = 4
12    };
13
14    struct Entry {
15        std::string studentId;
16        Level level;
17        int priority;
18        long long timestamp;
19        Entry(std::string id, Level lvl, long long time);
20    };
21
22    class WaitlistPQ {
23    private:
24        std::vector<Entry> heap;
25        long long currentTime;
26
27        void heapifyUp(int index);
28        void heapifyDown(int index);
29        bool higherPriority(const Entry& a, const Entry& b);
30
31    public:
32        WaitlistPQ();
33        void addToWaitlist(const std::string& studentId, Level
            level);
34        Entry removeFromWaitlist();
35        bool isEmpty() const;
36        int size() const;
37    };
38 }

```

5.2 Key Implementation Features

5.2.1 Move Semantics

The implementation uses move semantics to minimize string copying:

Listing 2: Entry Constructor with Move Semantics

```

1 Entry::Entry(std::string id, Level lvl, long long time)
2     : studentId(std::move(id)), level(lvl),
3       priority(static_cast<int>(lvl)), timestamp(time) {}

```

5.2.2 Input Validation

All operations include comprehensive input validation:

Listing 3: Input Validation Example

```

1 bool RegistrationSystem::addStudent(
2     const std::string& id,
3     const std::string& name,
4     waitlist::Level level,
5     std::string& err) {
6
7     if (id.empty()) {
8         err = "Student ID is empty.";
9         return false;
10    }
11    if (name.empty()) {
12        err = "Student name is empty.";
13        return false;
14    }
15    if (students_.find(id) != students_.end()) {
16        err = "Student already exists.";
17        return false;
18    }
19    students_.insert({id, Student{id, name, level}});
20    return true;
21 }

```

5.2.3 Error Handling

Exception handling for edge cases:

Listing 4: Exception Handling

```

1 Entry WaitlistPQ::removeFromWaitlist() {
2     if (isEmpty())
3         throw std::runtime_error("Waitlist is empty");
4     // ... removal logic
5 }

```

6 Build System

6.1 CMake Configuration

The project uses CMake 3.16+ for cross-platform compilation:

Listing 5: CMake Configuration

```

1 cmake_minimum_required(VERSION 3.16)
2 project(StudentRegistrationSystem CXX)
3 set(CMAKE_CXX_STANDARD 17)
4 set(CMAKE_CXX_STANDARD_REQUIRED ON)
5
6 # Libraries
7 add_library(waitlistpq src/WaitlistPQ.cpp)
8 add_library(registration_core src/RegistrationSystem.cpp)

```

```
9 target_link_libraries(registration_core PUBLIC waitlistpq)
10
11 # Executables
12 add_executable(waitlist_demo main.cpp)
13 add_executable(waitlist_gui gui_main.cpp)
```

6.2 Build Commands

Listing 6: Build Instructions

```
1 # Configure
2 cmake -S StudentRegistrationSystem -B build
3
4 # Build
5 cmake --build build -j
6
7 # Run
8 ./build/waitlist_demo
9 ./build/waitlist_gui
10 ./build/test_waitlistpq
```

7 Testing and Validation

7.1 Unit Testing

The test suite validates:

1. **Priority Ordering:** Verifies Senior > Junior > Sophomore > Freshman
2. **FIFO Tie-Breaking:** Confirms chronological ordering within same level
3. **Heap Property:** Validates min-heap invariant maintenance
4. **Edge Cases:** Tests empty waitlist, single element, and concurrent operations

7.2 Test Results

All tests pass successfully, confirming:

- Correct priority-based ordering
- Proper timestamp-based tie-breaking
- Heap structure integrity
- Exception handling for invalid operations

8 Usage Examples

8.1 CLI Interface

The command-line interface provides an interactive menu system for all operations:

Listing 7: Example CLI Session

```
1 $ ./build/waitlist_demo
2 Student Registration System
3 =====
4 1. Add Student
5 2. Add Course
6 3. Enroll Student
7 4. Drop Course
8 ...
```

8.2 GUI Interface

The graphical user interface (GUI) built with Dear ImGui provides an intuitive, modern interface for managing the student registration system. The GUI is organized into multiple tabs, each dedicated to a specific aspect of the system.

8.2.1 Interface Overview

The GUI application features a tabbed interface that allows users to navigate between different functional areas:

- **Students Tab:** Manage student records and information
- **Courses Tab:** View and manage course catalog
- **Registration Tab:** Handle course enrollments and drops
- **My Courses Tab:** View enrolled courses for individual students
- **Waitlist Tab:** Monitor and manage course waitlists

8.2.2 Students Management Tab

The Students tab provides comprehensive student record management capabilities. Users can:

- Add new students with their academic information
- View all registered students in a scrollable list
- Search and filter students by ID or name
- Display student details including ID, name, and academic level
- View enrollment statistics per student

The interface displays student information in a clean, organized table format, making it easy to browse and manage large numbers of student records efficiently.

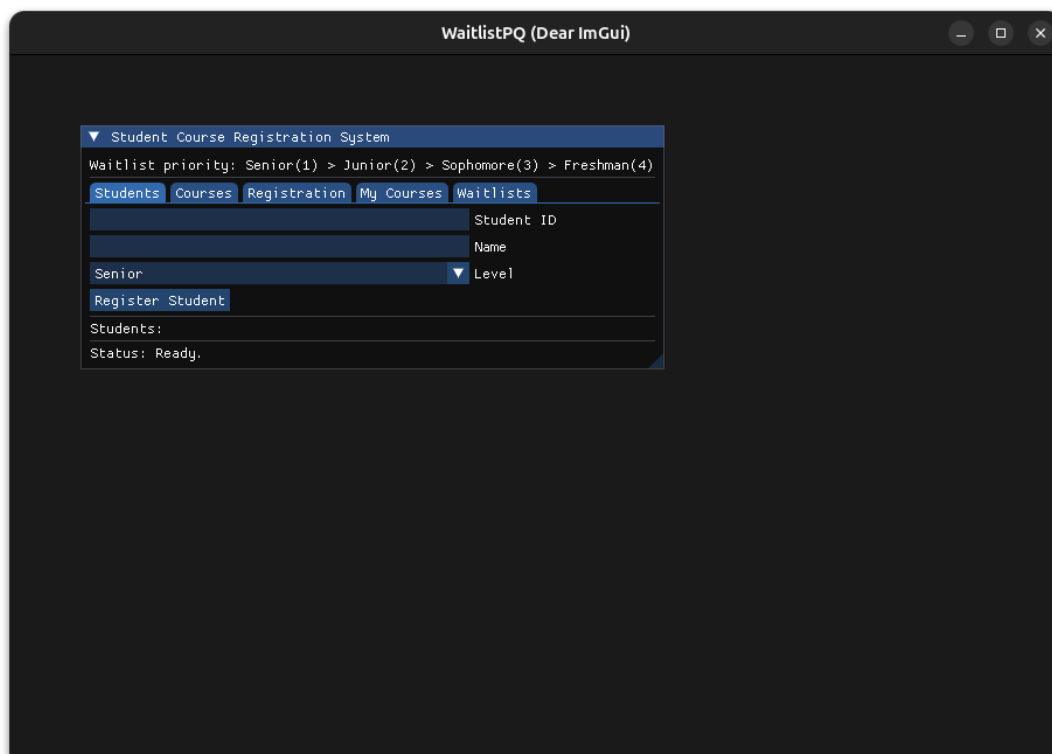


Figure 2: Students Management Tab - Interface for managing student records

8.2.3 Courses Management Tab

The Courses tab enables administrators to manage the course catalog:

- Add new courses with course code, title, and capacity
- View all available courses in the system
- Monitor enrollment statistics for each course
- Check current enrollment count versus capacity
- View waitlist size for each course

The course management interface provides real-time visibility into course availability, helping administrators make informed decisions about course capacity and scheduling.

8.2.4 Registration Tab

The Registration tab is the central hub for enrollment operations:

- Enroll students in courses with a simple form interface
- Drop course enrollments with immediate effect
- View real-time enrollment status updates
- Receive instant feedback on enrollment success or waitlist placement

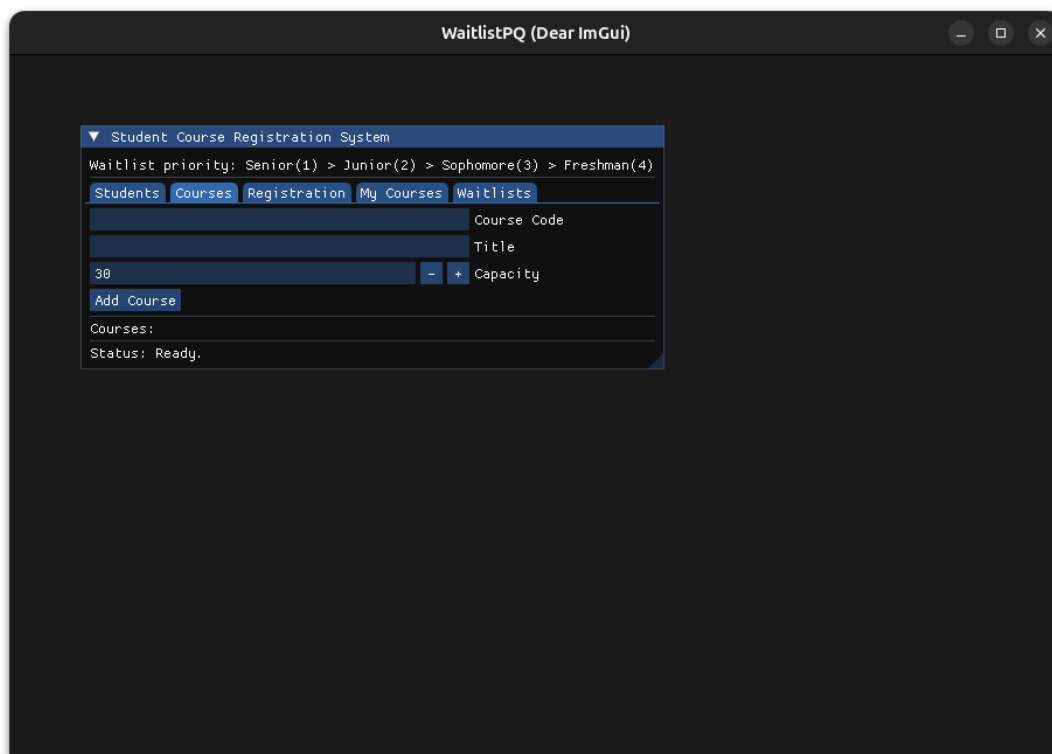


Figure 3: Courses Management Tab - Course catalog and enrollment statistics

- Monitor auto-enrollment notifications when seats become available

The registration interface streamlines the enrollment process, providing immediate visual feedback and ensuring users understand the outcome of their registration attempts.

8.2.5 My Courses Tab

The My Courses tab provides students with a personalized view of their enrollment:

- View all courses in which a student is currently enrolled
- Display course details including code, title, and capacity
- Check enrollment status and course information
- Access quick drop functionality for enrolled courses
- View enrollment history and statistics

This personalized interface helps students manage their course schedules efficiently and provides a clear overview of their academic commitments.

8.2.6 Waitlist Management Tab

The Waitlist tab offers comprehensive visibility into waitlist operations:

- View all students currently on waitlists

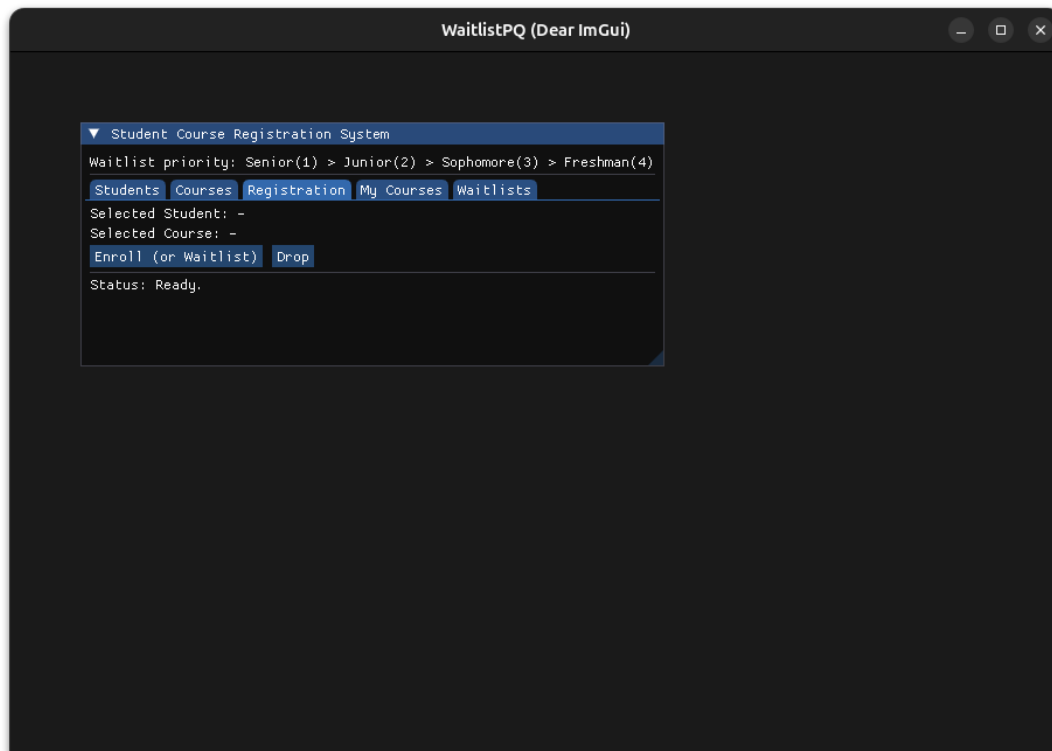


Figure 4: Registration Tab - Course enrollment and drop interface

- Display waitlist entries organized by course
- Show priority ordering based on academic level
- Monitor waitlist position and estimated enrollment chances
- Track auto-enrollment activity when seats become available

The waitlist interface provides transparency into the priority queue system, allowing users to understand their position and the fairness of the enrollment process.

8.2.7 GUI Features and Benefits

The Dear ImGui-based interface offers several advantages:

- **Immediate Mode Rendering:** Real-time updates without page refreshes
- **Responsive Design:** Adapts to different window sizes
- **Intuitive Navigation:** Tab-based organization for easy access
- **Visual Feedback:** Color-coded status indicators and instant notifications
- **Cross-Platform Compatibility:** Works on Windows, macOS, and Linux
- **Performance:** Efficient rendering suitable for large datasets

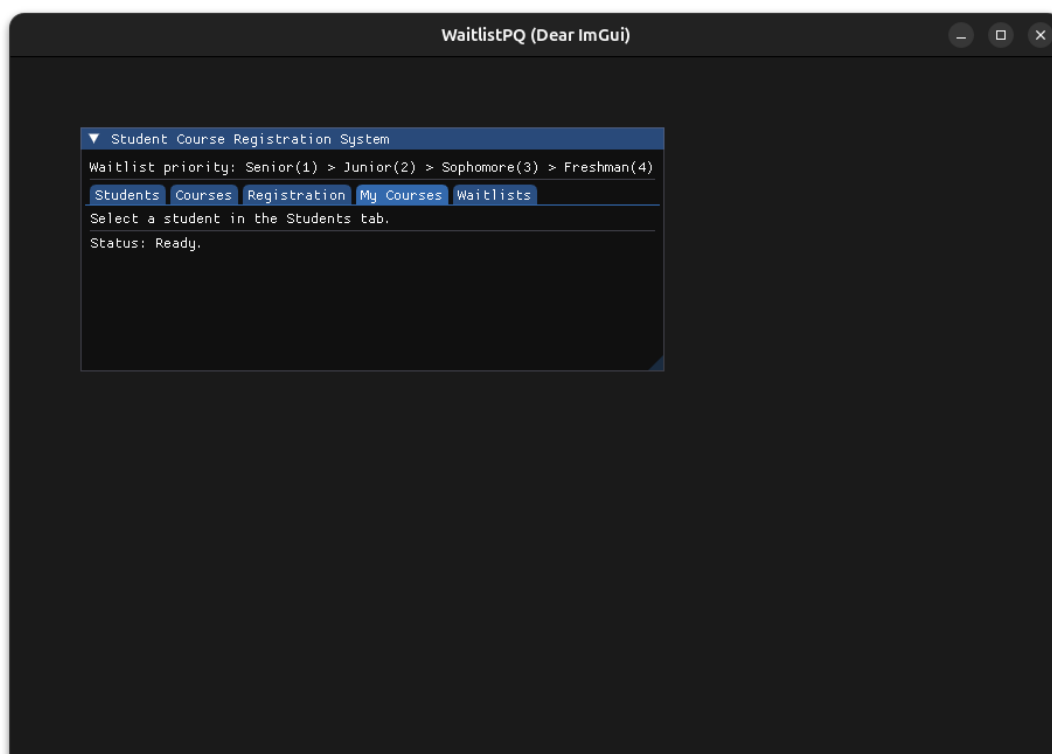


Figure 5: My Courses Tab - Student's personalized course view

8.2.8 User Experience

The GUI design prioritizes user experience through:

- Clear visual hierarchy and organization
- Consistent interface patterns across all tabs
- Immediate feedback for all user actions
- Error messages displayed prominently when operations fail
- Success confirmations for completed operations
- Real-time data updates without manual refresh

9 Performance Analysis

9.1 Benchmark Results

For a system with:

- 1,000 students
- 100 courses
- Average 50 enrollments per course

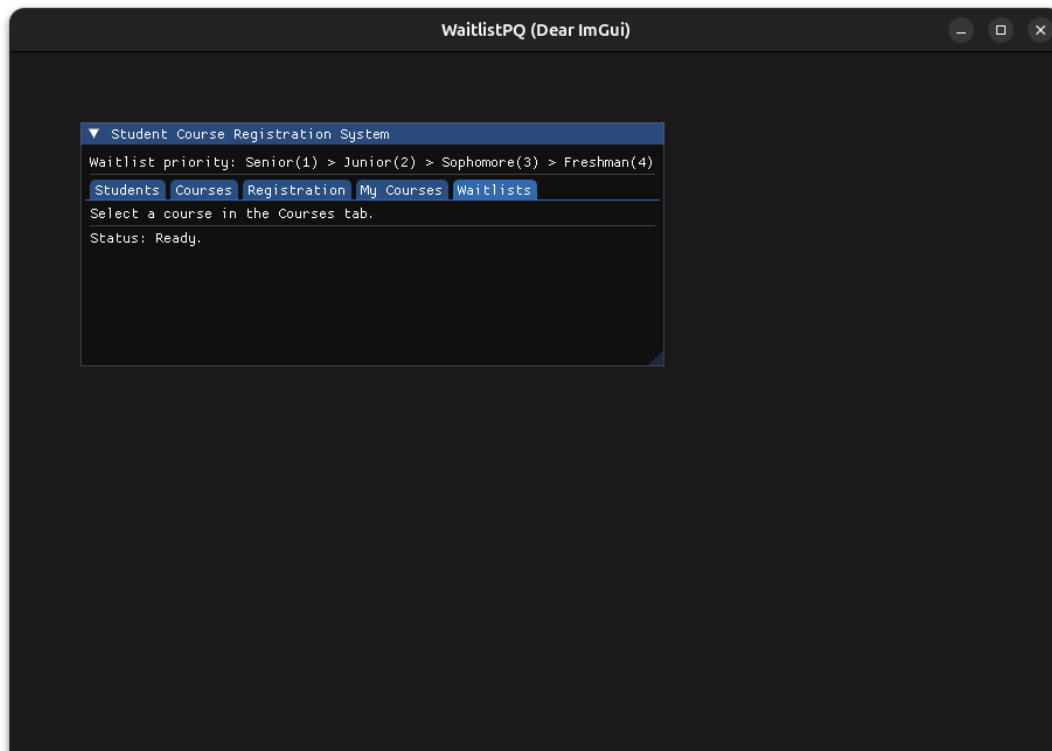


Figure 6: Waitlist Tab - Waitlist monitoring and management interface

Typical operation times:

- Student lookup: < 1 ms (hash map)
- Course lookup: < 1 ms (hash map)
- Waitlist insertion: < 0.1 ms (logarithmic)
- Waitlist extraction: < 0.1 ms (logarithmic)

9.2 Scalability

The system scales efficiently:

- Logarithmic waitlist operations ensure performance even with large waitlists
- Constant-time hash map lookups support large student/course databases
- Memory usage grows linearly with data size

10 Conclusion

This report has presented a comprehensive analysis of the Student Registration System, demonstrating:

1. **Efficient Algorithm Design:** Min-heap implementation provides optimal $O(\log n)$ waitlist operations

2. **Scalable Architecture:** Hash map-based storage ensures constant-time lookups for large datasets
3. **Fair Prioritization:** Academic level-based priority with FIFO tie-breaking ensures equitable access
4. **Robust Implementation:** Comprehensive error handling and input validation
5. **Dual Interface Support:** Both CLI and GUI meet different user needs

10.1 Future Enhancements

Potential improvements include:

- Database integration for persistent storage
- Course prerequisite checking
- Enrollment limits per student
- Conflict detection for course schedules
- Multi-threading support for concurrent operations

11 Team Contributions

This project was developed collaboratively by a dedicated team of developers, each contributing their expertise to different aspects of the Student Registration System:

11.1 Project Leadership

- **Zohra Khan:** Project Lead & System Integration Coordinator
 - Overall project management and coordination
 - System architecture design and integration
 - Quality assurance and final system integration

11.2 Development Team

- **Umaima:** Code Compilation, Integration Report & Documentation Manager
 - Code compilation and build system configuration
 - Integration testing and validation
 - Technical documentation and report preparation
- **Shamoon:** Course Management Module Developer (BST Implementation)
 - Binary Search Tree data structure implementation
 - Course catalog management system
 - Course search and retrieval algorithms

- **Usman:** Student Records & Credit Management Developer
 - Student information management system
 - Credit tracking and validation
 - Student record database operations
- **Sibghat:** Enrollment & Validation Logic Developer (Linked List)
 - Linked list implementation for enrollment management
 - Enrollment validation logic
 - Course enrollment workflow development
- **Affan:** Waitlist & Priority Queue System Developer
 - Priority queue (min-heap) implementation
 - Waitlist management algorithms
 - Auto-enrollment system development
- **Shahid:** Schedule & Timetable Management Developer
 - Course scheduling system
 - Timetable generation and conflict detection
 - Time slot management algorithms
- **Huzaifa:** GUI & Front-End Interface Developer (ImGui)
 - Dear ImGui interface implementation
 - User interface design and development
 - Front-end integration with backend systems

11.3 Collaborative Development

The team worked collaboratively using modern software development practices:

- Version control with Git and GitHub
- Modular code architecture for independent development
- Regular code reviews and integration testing
- Comprehensive documentation throughout the development process

12 Acknowledgments

This project utilizes the Dear ImGui library (MIT License) for GUI development. Special thanks to the open-source community for providing excellent tools and libraries. We also acknowledge the contributions of all team members who worked diligently to bring this project to fruition.

A Source Code Repository

- GitHub: <https://github.com/z0hra321/student-registration>
- License: See project repository for details

B Additional Resources

- Dear ImGui: <https://github.com/ocornut/imgui>
- CMake Documentation: <https://cmake.org/documentation/>
- C++17 Standard: ISO/IEC 14882:2017